

Архитектура робототехнических систем: ROS2, middleware, коммуникации

Замятин Дмитрий Александрович

- заведующий лаборатории инновационных технологий в робототехнике СУНЦ УрФУ
- доцент кафедры информатики СУНЦ УрФУ

ИИ-агенты: DeepSeek V4 Pro/Flash, GPT-5.5, Claude Sonnet 4/5, Minimax M3

Уровень 1: Лекция

Уровень 2: Практика

Уровень 3: Робот TIAGo (PAL robotix)

Как устроен курс (лекция + материал)

GitHub: https://github.com/DiZamer/ROS2_lecture_ya_camp.git



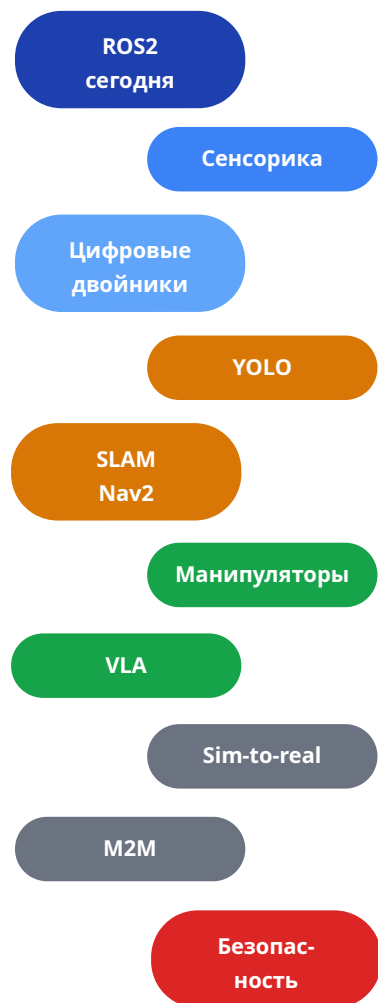
На слайдах — две ссылки: Ур.2 (практика) и Ур.3 (робот)

Директория	Назначение	Уровень
1_lecture/ 1_slides/ 1_demo/	План лекции, слайды, демонстрации	Ур.1
2_knowledge/ 2_practice/ 2_code/	Wiki-статьи, пошаговые задания, пакеты	Ур.2
2_cheatsheets/ .devcontainer/	Шпаргалки CLI, среда ROS2 Jazzy	Ур.2
3_Robot/ TIAgo_humble/	Архитектурный кейс — TIAgo (Humble+Gazebo)	Ур.3

Не запоминайте всё!!! - стройте карту: что за чем и почему



TIAGo — мобильный манипулятор PAL Robotics



1. МОДУЛЬНАЯ АРХИТЕКТУРА (модульная схема)



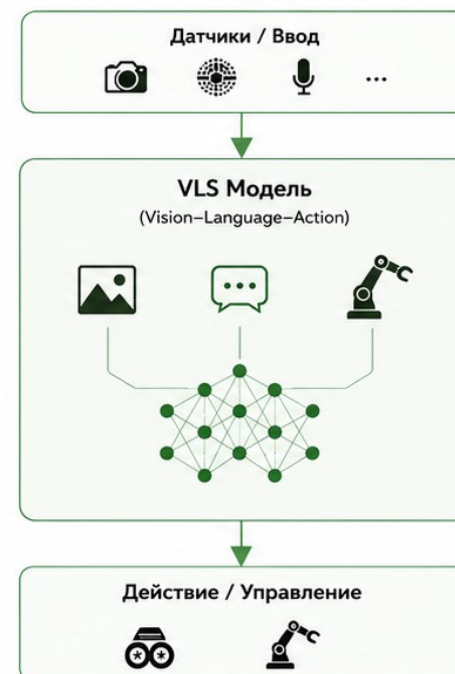
ПРЕИМУЩЕСТВА

- Чёткое разделение модулей
- Простота разработки и отладки
- Лёгкая замена и улучшение отдельных модулей
- Прозрачность и управляемость
- Хорошая масштабируемость отдельных компонентов

НЕДОСТАТКИ

- Сложность интеграции модулей
- Передача данных между модулями (задержки и накопление ошибок)
- Требует больших данных и вычислительных ресурсов
- Сложность обеспечения сквозной оптимизации
- Больше компонентов — больше точек отказа

2. СКВОЗНАЯ АРХИТЕКТУРА (VLS) (Vision–Language–Action)



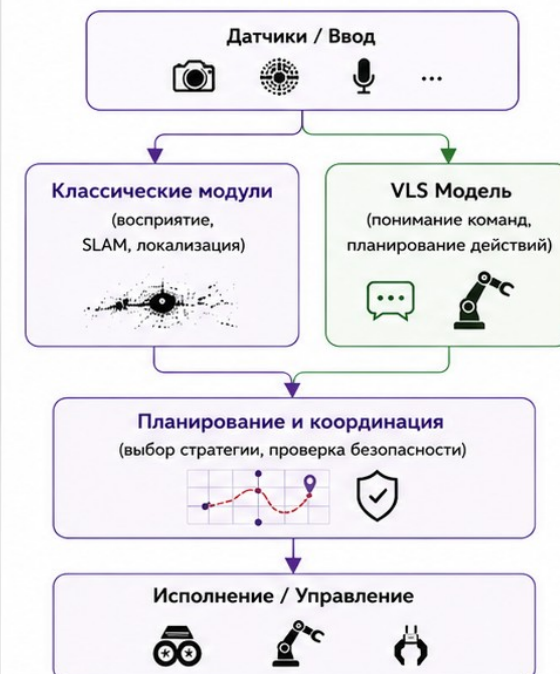
ПРЕИМУЩЕСТВА

- Единая модель «от входа до действия»
- Лучшее обобщение и понимание контекста
- Меньше ручных правил и логики
- Потенциал для более гибкого поведения
- Может использовать многоуровневую информацию совместно

НЕДОСТАТКИ

- Требует больших данных и вычислительных ресурсов
- Сложно интерпретировать и отлаживать
- Меньшая предсказуемость поведения
- Сложность обеспечения безопасности
- Данные и обучение дорогостоящи

3. ГИБРИДНАЯ АРХИТЕКТУРА (комбинированная схема)



ПРЕИМУЩЕСТВА

- Сочетает надёжность модульной схемы и гибкость VLS
- Критические задачи — через проверенные модули
- Сложные/неструктурированные задачи — через VLS
- Баланс производительности и безопасности
- Более плавный переход к VLS

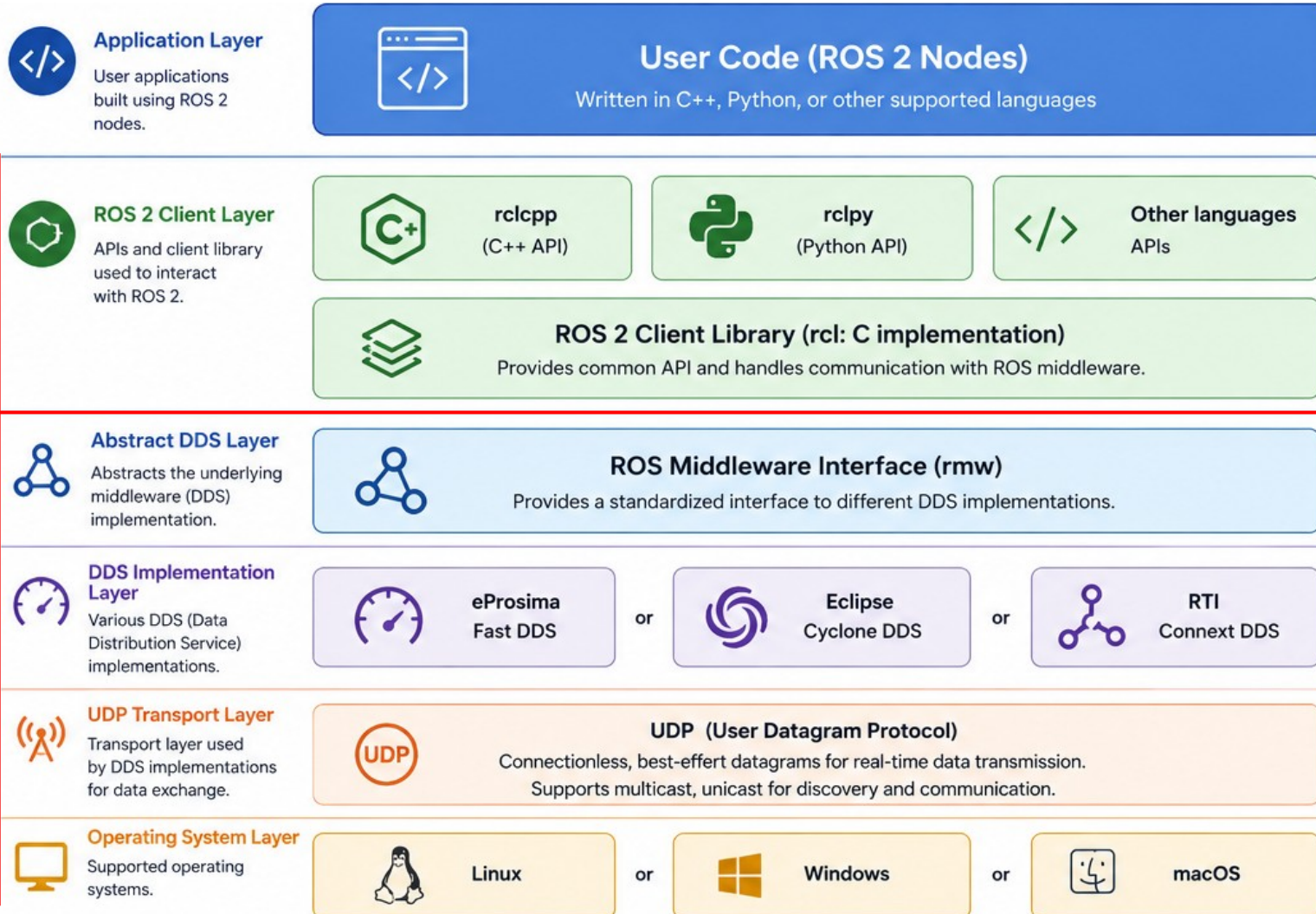
НЕДОСТАТКИ

- Сложность проектирования и интеграции
- Требуются обе компетенции: классические и ML/VLS
- Риски несогласованности между частями системы
- Больше компонентов — больше точек отказа

Что такое ROS2

4

ROS2 framework — среда для обмена сообщениям программ / слой между кодом и сетевым транспортом



Официальная документация: docs.ros.org



Что берёт на себя ROS2:

- Доставка сообщений между программами
- Discovery — авто-обнаружение узлов в сети
- Сериализация структур в байты
- QoS — надёжная или быстрая доставка
- tf2 — координатные преобразования
- Launch — запуск системы из многих узлов

Аналогия — городская инфраструктура:

- Дороги = DDS (транспорт сообщений)
- Почта = RMW (адаптер к службе доставки)
- Адреса = Topics (именованные каналы)
- Светофоры = QoS (правила движения)

Middleware: путь сообщения

- **DDS** — протокол передачи данных (сериализация, discovery, QoS)
- **RMW** — слой-адаптер между ROS2 API и конкретным DDS
- **Discovery** — авто-знакомство узлов
- **Программист пишет бизнес-логику** – ему не надо писать сетевой код, открывать сокеты, сериализовать сообщения, обрабатывать разрывы соединения.

Путь сообщения (7 шагов):

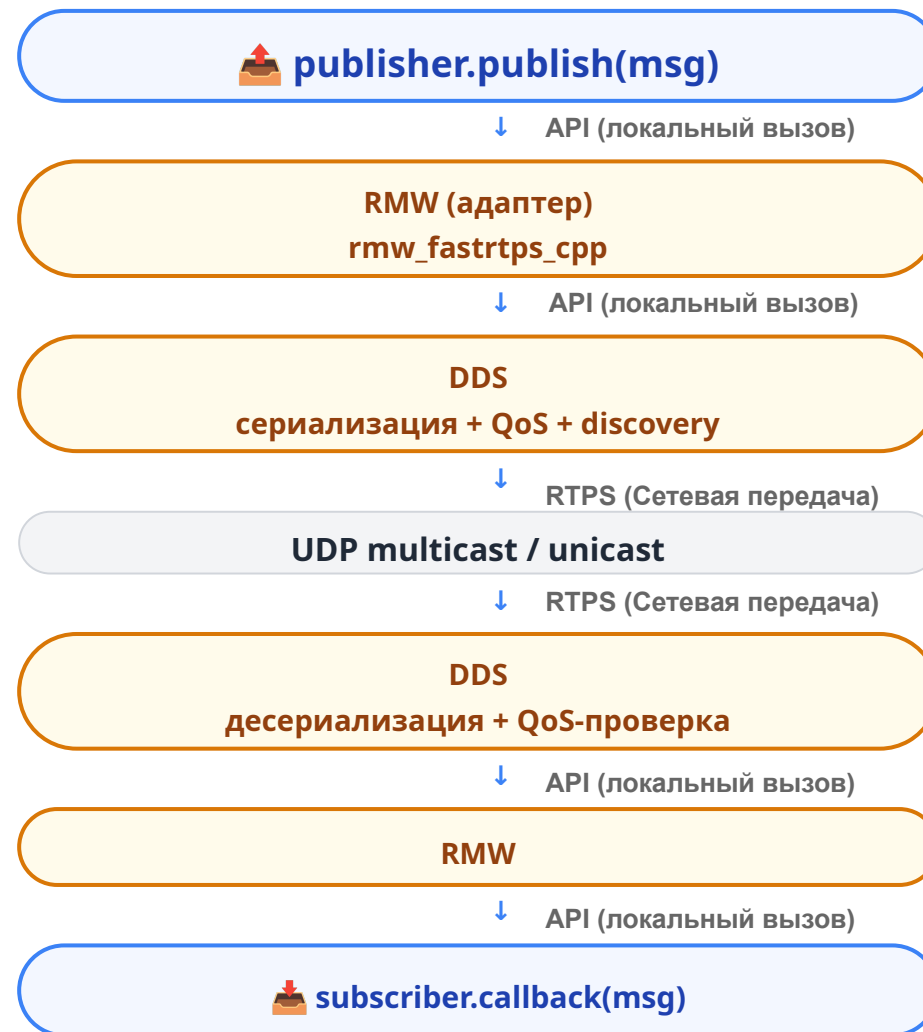
1. Вы вызываете `publisher.publish(msg)`
2. RMW переводит вызов в API конкретного DDS
3. DDS сериализует структуру в байты
4. QoS-контроль проверяет настройки доставки
5. UDP multicast (discovery) / unicast (данные) в сеть
6. DDS получателя десериализует сообщение
7. RMW передаёт его в ROS2 → ваш callback

```
# Диагностика middleware
ros2 doctor – report
```

```
# сменить: export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
printenv RMW_IMPLEMENTATION
```

Уп.2: [knowledge/ros_architecture.md](#) · [dds_protocol.md](#)

Уп.3: Robot/ TIAGo использует CycloneDDS



ROS Graph

6

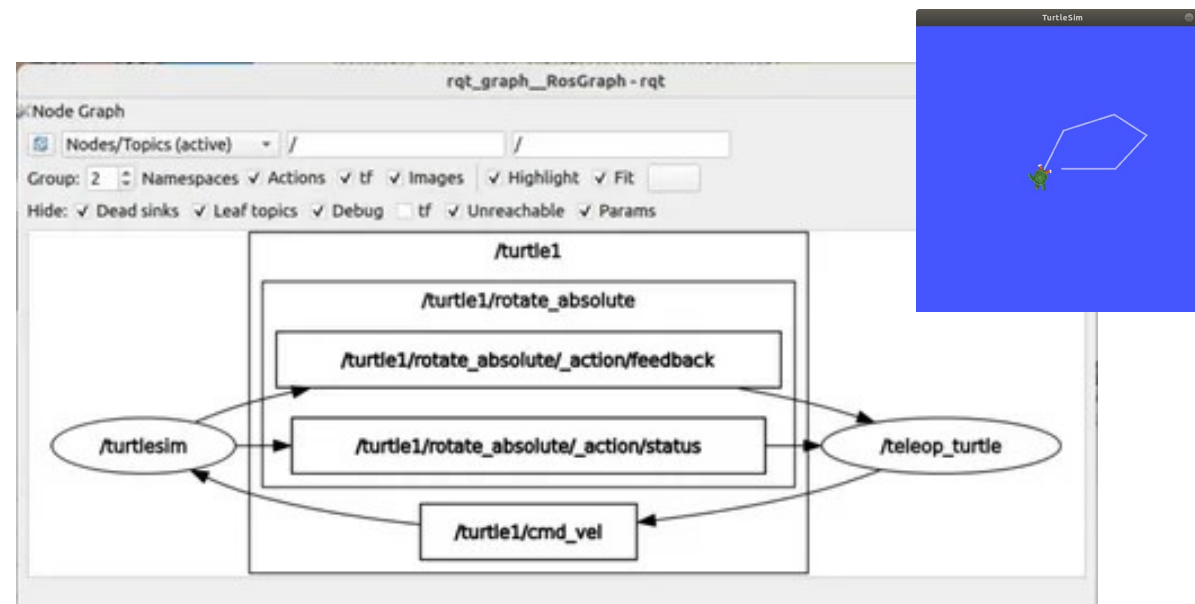
- **Node (узел)** — программа с одной задачей
- **ROS Graph** — карта узлов и связей
- **Executor** — цикл событий, вызывает callbacks
- **rqt_graph** — визуализация графа

Три способа связи:

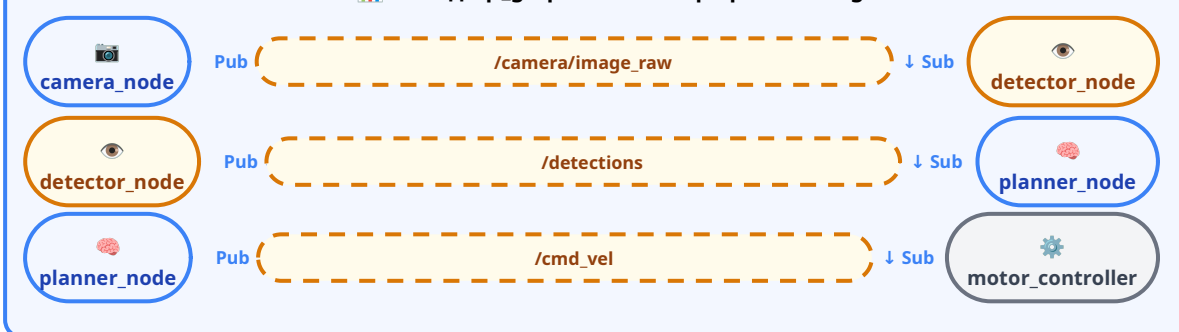
- **Topic** → **поток**, много→много (Telegram-канал)
- **Service** → **запрос-ответ**, 1→1 (звонок)
- **Action** → **долгая задача**, прогресс, отмена (доставка)

Executor: цикл, который ждёт события (таймер, сообщение, запрос) и вызывает соответствующий callback

```
# CLI для работы с графом
ros2 node list           # все запущенные узлы
ros2 node info /my_node  # подписки, публикации, сервисы
rqt_graph                # визуализация графа
```



Вывод rqt_graph — ROS Graph работа TIAgo



Подсистемы работа

Подсистема — это группа ROS2-узлов с общей зоной ответственности, стандартными входами/выходами и чёткими границами

 **Sensor Suite**
LiDAR, camera, IMU → topics

 **Mobile Base**
motor controller, /cmd_vel, /odom

 **Navigation — **Action****
Nav2: SLAM, planner, controller

 **Manipulation — **Action****
MoveIt2: IK, OMPL, ros2_control

 **Perception**
YOLO: /camera → /detections

 **Safety — **Service****
E-stop, watchdog, battery

 **LLM Bridge**
voice → policy → safety → action

Подсистема	Вход	Выход	Интерфейс
Sensor Suite	—	/scan, /image_raw	Topic
Mobile Base	/cmd_vel	/odom	Topic
Navigation	/scan, /odom, карта	/cmd_vel	Action
Manipulation	Joint states	trajectory	Action
Perception	/image_raw	/detections	Topic
Safety	/battery, /detections	/emergency_stop	Service
LLM Bridge	/voice_command	goal Nav2/MoveIt2	Action

Группа узлов + общий интерфейс. Меняйте внутренности подсистемы — интерфейс остаётся. TIAGo: 8 подсистем · 40+ пакетов · 20+ topics

Контейнерный подход



ROS2 ставится на хост **и в контейнер**

Преимущества контейнеризации:

- Одинаковое окружение у всех разработчиков
- Изоляция — ROS2 не конфликтует с программами хоста
- Воспроизводимость — контейнер пересоздается за минуты
- Dev Container: Ubuntu 24.04 + ROS2 Jazzy

```
# Проверка окружения
ros2 --help          # справка по CLI
ros2 run demo_nodes_cpp talker  # тестовый узел
ros2 doctor --report  # диагностика
printenv RMW_IMPLEMENTATION  # текущий DDS
```

Хост: Windows / macOS / Linux

↓ Docker Engine

Dev Container
Ubuntu 24.04 + ROS2 Jazzy

или

Ур.3 контейнер
Ubuntu 22.04 + Humble + Gazebo

Workspace + Package + colcon

Workspace — корневая папка проекта:

```
~/ros2_ws/
├── src/      ← исходники пакетов
├── build/    ← промежуточные файлы
├── install/  ← готовые артефакты
└── log/      ← логи сборки
```

Package — папка со строгой структурой:

```
my_first_pkg/
├── package.xml      # имя, версия, зависимости
├── setup.py         # entry_points → console_scripts
├── setup.cfg
├── resource/ + __init__
└── my_first_pkg/
    └── my_node.py   # код узла
```

```
# Создать пакет
ros2 pkg create --build-type ament_python my_pkg \
  --dependencies rclpy std_msgs
```

package.xml — метаданные и зависимости:

```
<package format="3">
  <name>my_first_pkg</name>
  <version>0.0.1</version>
  <buildtool_depend>ament_python</buildtool_depend>
  <depend>rclpy</depend>
  <depend>std_msgs</depend>
</package>
```

setup.py — точка входа в узел:

```
entry_points={
    'console_scripts': [
        'my_node = my_first_pkg.my_node:main',
    ],
},
```



colcon build — сборка всех пакетов одной командой

--symlink-install — изменения Python-кода видны без пересборки

rosdep install — установка зависимостей из package.xml

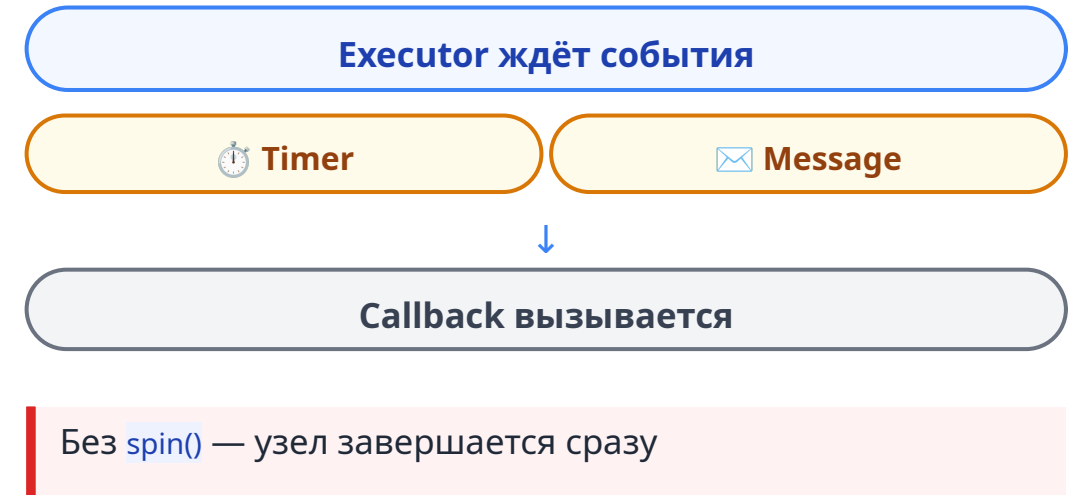
Node + spin() + Executor

```
import rclpy # ROS 2 клиент для Python
from rclpy.node import Node # Базовый класс узла

class MyNode(Node): # Создаем свой узел
    def __init__(self): # Конструктор
        super().__init__('my_node') #Родительский узел
        self.create_timer(1.0, self.cb) # Таймер
    def cb(self): # Функция обратного вызова
        self.get_logger().info('Tick') # Выводв лог

def main(args=None): # Главная функция
    rclpy.init(args=args) # Инициализируем ROS 2
    node = MyNode() # Создаем узел
    rclpy.spin(node) # Запускаем Executor
    node.destroy_node() # Уничтожаем узел
    rclpy.shutdown() # Завершаем ROS 2

if __name__ == '__main__': # Если запущен напрямую
    main() # Запускаем main
```



Topic: Publisher + Subscriber

Publisher — узел, который отправляет сообщения в topic:

```
from std_msgs.msg import String

class Talker(Node):
    def __init__(self):
        super().__init__('talker')
        # create_publisher(тип, имя_топика, глубина)
        self.publisher = self.create_publisher(
            String, '/chatter', 10)
        self.timer = self.create_timer(
            1.0, self.timer_callback)

    def timer_callback(self):
        msg = String()
        msg.data = f'Hello {self.count}'
        self.publisher.publish(msg)
        self.count += 1
```

Subscriber — подписывается и получает сообщения:

```
class Listener(Node):
    def __init__(self):
        super().__init__('listener')
        # create_subscription(тип, имя, callback, глубина)
        self.subscription = self.create_subscription(
            String, '/chatter', self.callback, 10)

    def callback(self, msg):
        self.get_logger().info(
            f'I heard: {msg.data}')
```



Pub и sub не знают друг о друге — только имя топика и тип сообщения. Discovery сводит их автоматически. Множество sub'ов читают одновременно.

Topic — это именованный канал с фиксированным типом сообщений, просто метка, который не создаётся отдельно. Появляется, когда первый publisher начинает публиковать.

Callback вызывается при каждом сообщении. Должен быть быстрым — иначе потеряете сообщения.

Уп.2: [knowledge/topics.md](#) · [demo/demo2_topics.md](#) · [cheatsheets/ros2_cli.md](#)

Уп.3: [Robot/ ros2 topic echo /scan](#) · [ros2 topic hz /odom](#)

```
# Список активных topics
ros2 topic list

# Чтение в реальном времени
ros2 topic echo /chatter

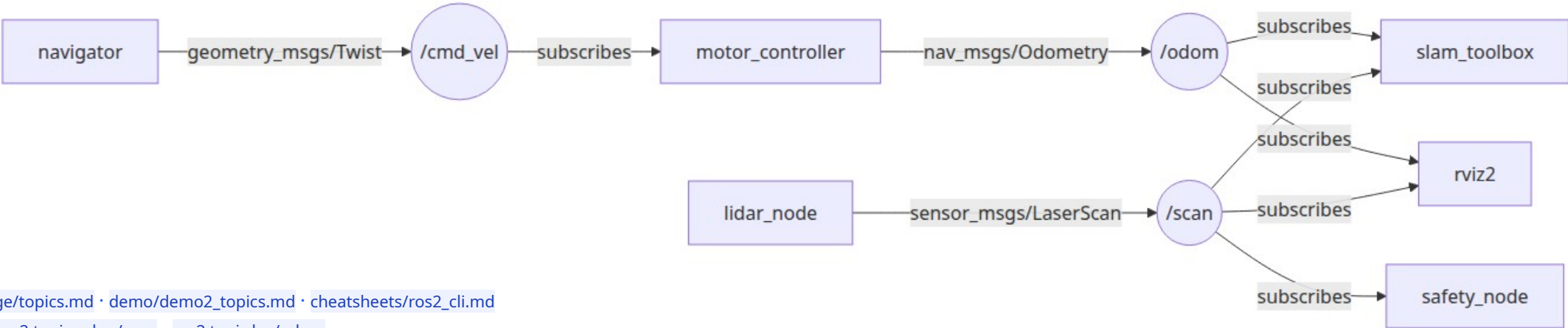
# Публикация вручную
ros2 topic pub /cmd_vel geometry_msgs/Twist \
  "{linear: {x: 0.5}, angular: {z: 0.0}}"

# Частота и статистика
ros2 topic hz /scan      # частота публикации
ros2 topic bw /chatter   # пропускная способность
ros2 topic info /scan    # тип + кто pub/sub
```

Типы сообщений (стандартные):

Тип	Поля	Где используется
std_msgs/String	data	текст, отладка
geometry_msgs/Twist	linear, angular	/cmd_vel
sensor_msgs/LaserScan	ranges[], angle_min	/scan
sensor_msgs/Image	data[], width, height	/camera/image_raw
nav_msgs/Odometry	pose, twist	/odom

Реальные topics робота (TIAGo):



Service: Запрос → Ответ

13

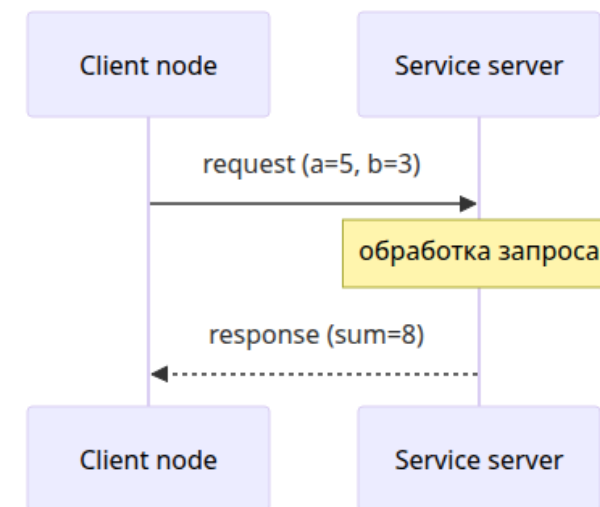
Service — механизм для короткой операции, где один узел (client) отправляет запрос, а другой (server) отвечает

Server — обрабатывает запрос:

```
self.srv = self.create_service(
    AddTwoInts, '/add_two_ints', self.cb)
def cb(self, req, resp):
    resp.sum = req.a + req.b
    return resp
```

Client — вызывает сервис:

```
self.cli = self.create_client(
    AddTwoInts, '/add_two_ints')
while not self.cli.wait_for_service(
    timeout_sec=1.0):
    self.get_logger().info('waiting...')
future = self.cli.call_async(request)
future.add_done_callback(
    self.response_callback)
```



Сервисы в работе TIAGo:

Service	Тип	Назначение
<code>/emergency_stop</code>	Trigger	Аварийная остановка
<code>/reset_motors</code>	Trigger	Сброс ошибки контроллера
<code>/enable_lidar</code>	SetBool	Вкл/выкл лидар

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 5, b: 3}"
```

Важно: Client ждёт server через `wait_for_service()`. Без этого — исключение.
Service — это **ВЫЗОВ**, а не подписка.

Уп.2: [knowledge/services.md](#) · [demo/demo3_services.md](#)

Уп.3: Robot/ Topic — сенсоры · Service — `emergency_stop`, `reset_motors`

Критерий	Topic	Service
Связь	Многие → многие	1 → 1
Направление	Однонаправленный поток	Запрос → Ответ
Ответ	Нет	Один ответ
Частота	Высокая (10–100 Гц)	Низкая (по запросу)
Гарантия доставки	Через QoS	Неявная (есть ответ = доставлено)
Пример	/scan, /cmd_vel	/emergency_stop, /reset_motors

Поток данных, много получателей → topic · Команда с подтверждением → service

Topic — радио: вещает всем, без ответа	Service — телефонный звонок: один на один, с ответом
--	--

Action: долгая задача

Action — механизм для длительной задачи, у которой есть цель (goal), прогресс (feedback), итог (result) и возможность отмены (cancel)

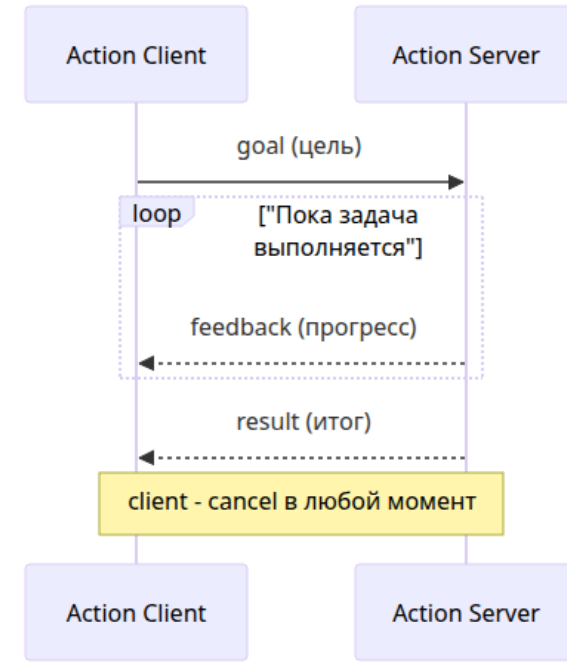
- **Goal** — цель («доставь пиццу Маргарита»)
- **Feedback** — прогресс («выехал», «подъезжаю»)
- **Result** — итог («доставлено» / «не удалось»)
- **Cancel** — отмена (в любой момент)

Внутри action — 3 топика + 3 сервиса:

```
.../_action/send_goal      — service: отправить цель
.../_action/get_result     — service: получить итог
.../_action/cancel_goal    — service: отмена
.../_action/feedback       — topic: промежуточный прогресс
.../_action/status         — topic: статус цели
```

Callbacks action client:

- `feedback_callback` — прогресс
- `goal_response_callback` — цель принята/отклонена
- `result_callback` — финальный результат



Примеры action в работе:

Action	Feedback	Отмена
<code>/navigate_to_pose</code>	Оставшееся расстояние	✓
MoveIt2 trajectory	Прогресс траектории	✓
<code>/follow_path</code>	Текущая точка пути	✓

```
ros2 action send_goal /fibonacci example_interfaces/action/Fibonacci "{order: 5}" --feedback
```

Topic vs Service vs Action

16

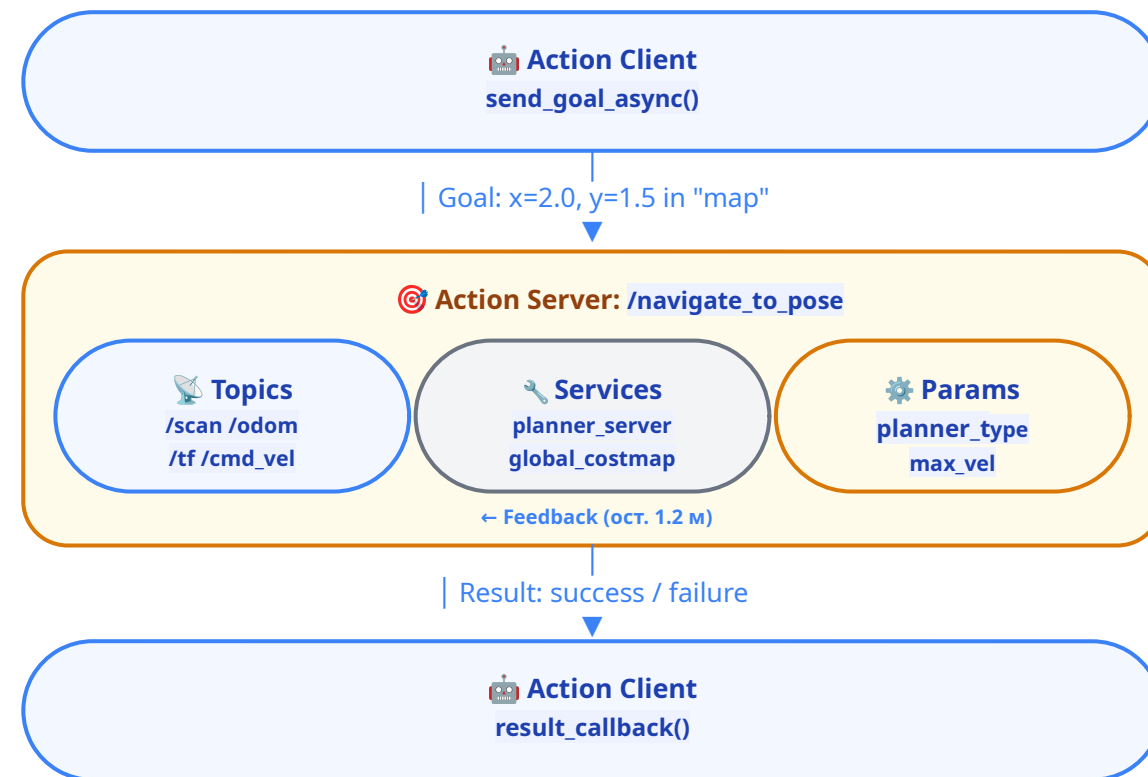
Критерий	Topic	Service	Action
Связь	M→M	1→1	1→1
Ответ	Нет	Один ответ	Прогресс + итог
Длительность	Непрерывно	Мгновенно (< 1 сек)	Секунды / минуты
Отмена	Нет	Нет	Есть
Типичный сценарий	Поток сенсора	Команда управления	Навигация / манипуляция
Пример	/scan (10 Гц)	/emergency_stop	/navigate_to_pose

Поток → Topic

Запрос-ответ → Service

Долгая задача → Action

Анатомия Action — Nav2:



Action = **оркестратор** topics + services + parameters

Ур.2: [knowledge/actions.md](#) · [cheatsheets/topics_services_actions.md](#)

Ур.3: Robot/ все три механизма в архитектуре TIAGo

Quiz: Topic, Service или Action?

17

- 1 Данные лидара, 10 раз в сек, читают 3 узла одновременно
- 2 Кнопка emergency stop с подтверждением срабатывания
- 3 Доехать из кухни в спальню с прогрессом и возможностью отмены

Часть 2

Launch, Parameters, tf2, QoS, Lifecycle, Simulation, Архитектурные мосты

Параметр — настройка узла, меняется без перекомпиляции

```
class MyNode(Node):
    def __init__(self):
        super().__init__('my_node')
        # объявить параметры с default-значением
        self.declare_parameter('publish_rate', 1.0)
        self.declare_parameter('frame_id', 'lidar')
        # читать в callback (не в __init__!)
        rate = self.get_parameter('publish_rate').value
```

YAML-конфиг:

```
# my_package/config/params.yaml

my_node:
  ros__parameters:
    publish_rate: 5.0
    frame_id: "camera_link"
```

```
ros2 run my_package my_node --ros-args --params-file config/params.yaml
```

Ур.2: [knowledge/parameters.md](#)

Ур.3: [3_Robot/TIAgo_humble/docs/launch_params.md](#)

```
# CLI-команды
ros2 param list           # список параметров узлов
ros2 param get /node rate # прочитать
ros2 param set /node rate 5.0 # изменить на лету!
ros2 param dump /node     # сохранить в YAML
ros2 param load /node params.yaml # загрузить из YAML
```

Пример: узел с настраиваемой частотой:

```
class TimedTalker(Node):
    def __init__(self):
        super().__init__('timed_talker')
        self.declare_parameter('publish_rate', 1.0)
        self.declare_parameter('msg_prefix', 'Hello')
        rate = self.get_parameter('publish_rate').value
        period = 1.0 / rate if rate > 0 else 1.0
        self.timer = self.create_timer(period, self.cb)

    def cb(self):
        prefix = self.get_parameter('msg_prefix').value
        self.get_logger().info(f'{prefix} #{self.count}')
```

Читайте `get_parameter()` в callback,
не в `__init__` — иначе `ros2 param set` не работает

Launch

Launch — сценарий запуска нескольких узлов одной командой: какие узлы, с какими параметрами, в каком порядке, с какими аргументами

```
# launch/my_launch.launch.py
# config — путь my_pkg/config/params_talker.yaml
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='my_pkg', executable='talker',
            name='talker', parameters=[config]),
        Node(package='my_pkg', executable='listener'),
    ])
```

С аргументами командной строки:

```
rate_arg = DeclareLaunchArgument(
    'publish_rate', default_value='1.0')
...
parameters=[{
    'publish_rate':
        LaunchConfiguration('publish_rate')
}]
```

Структура пакета с launch:

```
my_pkg/
├── my_pkg/                                # основной код (Python модуль)
│   ├── __init__.py
│   ├── talker.py
│   └── listener.py
├── launch/                               # launch файлы
│   ├── my_launch.launch.py
│   └── another.launch.py
├── config/                               # конфигурационные файлы
│   ├── params_talker.yaml
│   └── params_listener.yaml
├── resource/                             # ресурсы (обязательно!)
│   └── my_pkg
├── test/                                 # пустой файл с именем пакета
├── package.xml                           # тесты (опционально)
├── setup.py
├── setup.cfg                             # где вы описываете data_files
└──
```

CLI:

```
# Запуск
ros2 launch my_pkg demo.launch.py

# Проверка синтаксиса (без запуска)
ros2 launch -s my_pkg demo.launch.py

# Показать аргументы
ros2 launch my_pkg demo.launch.py --show-args
```

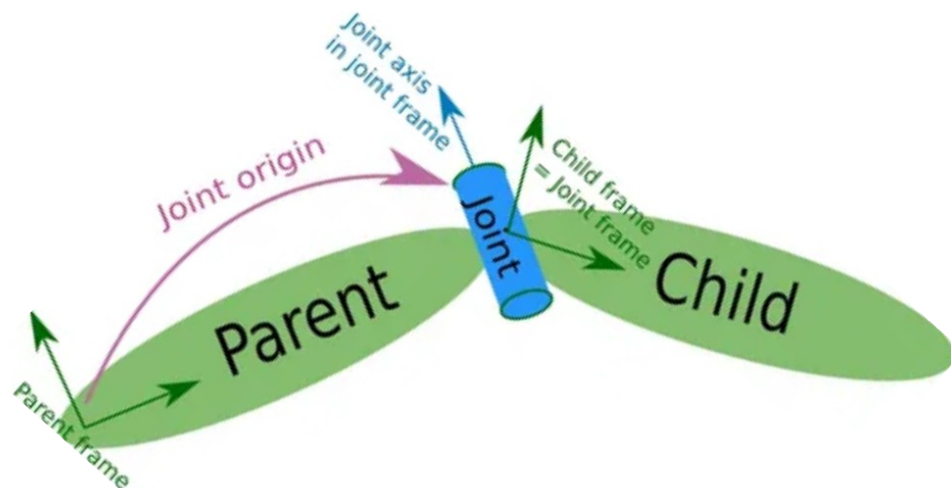
setup.py — добавляем в data_files:

```
data_files=[
    ('share/.../launch',
     glob('launch/*.launch.py')),
    ('share/.../config',
     glob('config/*.yaml')),
]
```

```
ros2 launch my_pkg my_launch.launch.py publish_rate:=5.0
```

Tf2 (Transforms): дерево координат

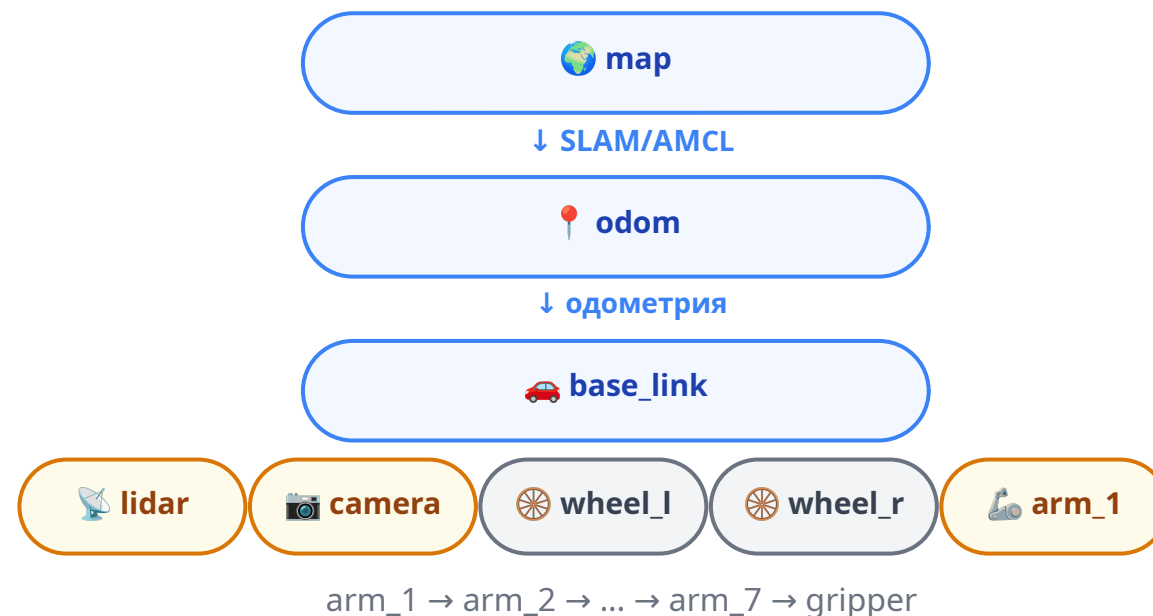
tf2 — это библиотека преобразований, которая позволяет отслеживать несколько координатных систем робота во времени



- **Frame** — система координат: `map`, `base_link`, `lidar_link`
- **Transform** — переход x, y, z + поворот (кватернион) – 7 чисел
- **Дерево** — каждый frame имеет ровно **одного родителя**
- **Зачем:** лидар видит препятствие в `lidar_link`, навигатору нужно в `map` — tf2 вычисляет цепочку

```
# Чтение transform
t = buffer.lookup_transform(
    'map', 'lidar_link', rclpy.time.Time())
```

Дерево координат:



```
# CLI
ros2 run tf2_ros tf2_echo map base_link # transform в реальном времени
ros2 run tf2_tools view_frames           # PDF дерева
ros2 topic echo /tf_static               # статические transforms
```

Без tf2 навигация невозможна. Nav2, MoveIt2, rviz2 — все зависят от tf2-дерева.

TF-дерево TIAGo включает ~25 фреймов

```
map (глобальная система координат – SLAM/AMCL)
├── odom (одометрия – DiffDriveController или DLO)
│   ├── base_footprint (проекция робота на пол – Nav2 base_frame_id)
│   │   ├── base_link (корневой фрейм робота)
│   │   │   ├── torso_lift_link (выдвижной топс)
│   │   │   │   ├── head_1_link → head_2_link (голова, pan/tilt)
│   │   │   │   │   ├── camera_link (RGB-D камера)
│   │   │   │   │   └── arm_1_link → arm_7_link (манипулятор 7-DOF)
│   │   │   │   │       └── wrist_ft_link → gripper_link (эндектор)
│   │   └── base_laser_link (лазерный сканер)
```

Важно: дерево, не граф - у каждого frame ровно один родитель, циклы запрещены.

Static transforms (из URDF, не меняются):

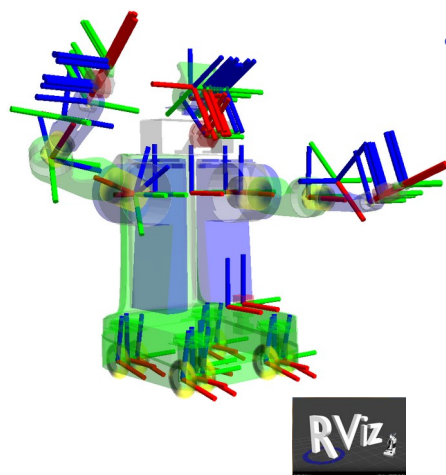
- `base_link → lidar_link`, `camera_link`, `wheel_*`
- Публикует `robot_state_publisher`

Dynamic transforms (меняются при движении):

- `odom → base_link` — одометрия от контроллера
- `map → odom` — SLAM корректирует положение

Кто публикует transforms:

Transform	Кто	Тип
<code>base → lidar</code>	<code>robot_state_publisher</code>	Static → топик /tf_static
<code>base → camera</code>	<code>robot_state_publisher</code>	Static → топик /tf_static
<code>odom → base</code>	Контроллер моторов	Dynamic → топик /tf
<code>map → odom</code>	SLAM / AMCL	Dynamic → топик /tf



Проверка:

```
# генерация PDF с деревом
ros2 run tf2_tools view_frames
```

```
# проверить конкретный transform
ros2 run tf2_ros tf2_echo \
  map gripper_link
```

QoS (Quality of Service): качество доставки

QoS для TIAGo

Topic	Reliability	Durability	Depth	Почему
<code>/scan</code>	Best effort 🟡	Volatile	10	Поток данных, потеря кадра не критична
<code>/cmd_vel</code>	Reliable 🟢	Volatile	10	Нельзя терять команды управления
<code>/odom</code>	Reliable 🟢	Volatile	10	Навигация зависит от точной одометрии
<code>/camera/image_raw</code>	Best effort 🟡	Volatile	5	Высокая частота, потеря кадра не критична
<code>/detections</code>	Reliable 🟢	Volatile	10	Нельзя терять факт обнаружения объекта
Карта (map)	Reliable 🟢	Transient local	1	Новый узел должен сразу получить карту

```
qos_scan = QoSProfile(depth=10, reliability=ReliabilityPolicy.BEST_EFFORT) # Задали правила
```

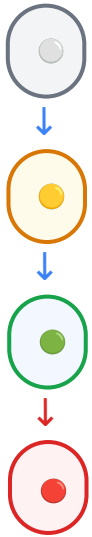
```
self.publisher = self.create_publisher(LaserScan, '/scan', qos_scan) # Задали qos_scan
```

Разные QoS у pub и sub → соединения не будет

Lifecycle: состояния узла

Lifecycle node — узел, который проходит через заданные состояния: **создание, настройка, активация, остановка, завершение**

LifecycleNode- запуск автомобиля:



- **on_configure()** — Вставили ключ: инициализация драйверов, память выделена, двигатель выключен.
- **on_activate()** — Повернули ключ на старт: двигатель запущен, исполнители включены.
- **on_active()** — Машина едет: обработка /cmd_vel, управление педалями.
- **on_deactivate()** — Глушим двигатель: моторы остановлены, ресурсы сохранены.
- **on_cleanup()** — Вынули ключ: полная очистка памяти и портов.
- **on_shutdown()** — Закрыли дверь и ушли: финальное завершение узла.

LifecycleNode - это Node с заданием состояния

- Состояния задаются явно через launch или CLI – не `roslaunch`
- Проходят последовательно все состояния и только в active узел полноценно работает

```
import rclpy
from rclpy.lifecycle import LifecycleNode, State
from rclpy.lifecycle import TransitionCallbackReturn
```

```
class ManagedNode(LifecycleNode):
```

```
    def __init__(self):
        super().__init__('managed_node')

    def on_configure(self, state: State) -> TransitionCallbackReturn:
        self.get_logger().info('Configuring...')
        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state: State) -> TransitionCallbackReturn:
        ...

    def on_deactivate(self, state: State) -> TransitionCallbackReturn:
        ...

    def on_cleanup(self, state: State) -> TransitionCallbackReturn:
        ...

    def on_shutdown(self, state: State) -> TransitionCallbackReturn:
        ...
```

```
# Transitioning state: Configuring → Inactive
ros2 lifecycle set /managed_node configure
```

Принцип: сначала цифровой двойник в симуляторе, потом реальное железо



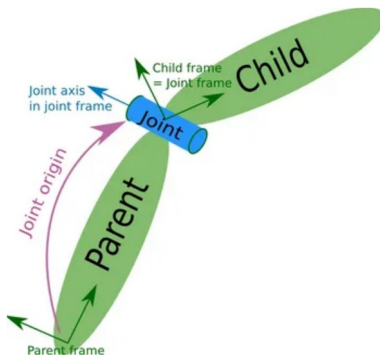
Почему simulation-first:

- ✓ **Безопасность** — ошибка в симуляции бесплатна.
На работе — сломанный сервопривод.
- ⚡ **Скорость** — симуляцию можно ускорить,
поставить на паузу, перезапустить.
- 🔄 **Воспроизводимость** — одинаковые условия:
нет «сегодня лидар глючит из-за солнца».
- 💻 **Разработка без железа** — робот стоит дорого,
симуляция бесплатно.

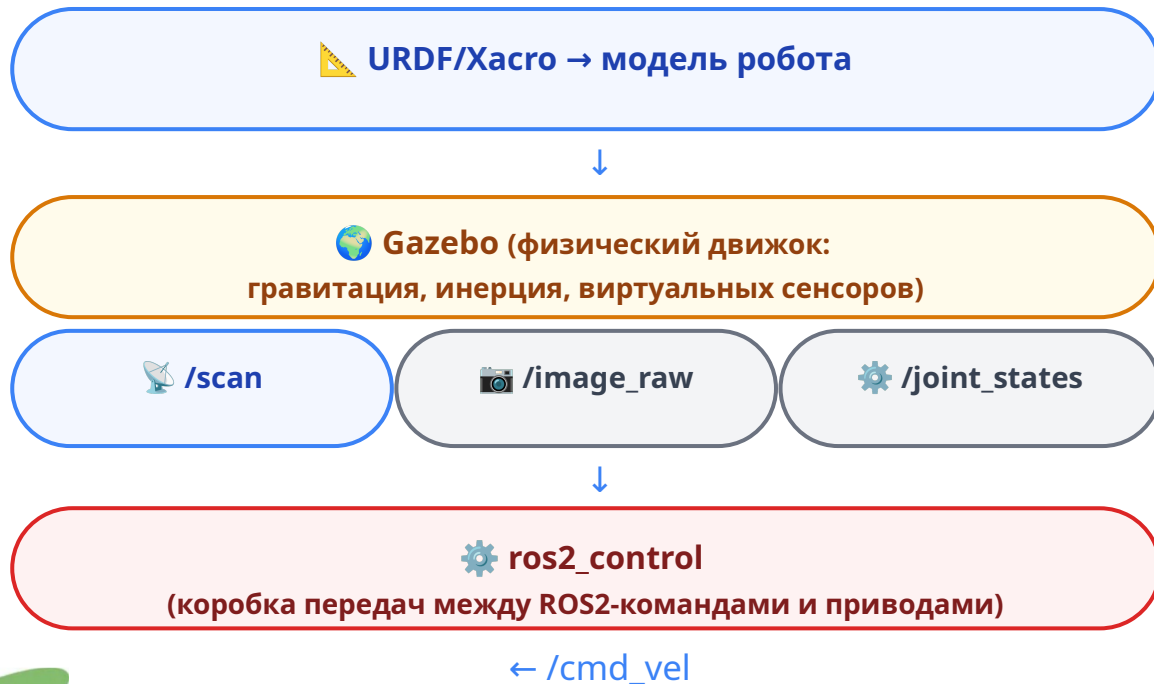
URDF (Unified Robot Description Format) — техпаспорт робота с геометрией на XML:

```
<link name="base_link">
  <visual>
    <geometry><box size="0.4 0.3 0.2"/></geometry>
  </visual>
</link>

<joint name="wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="wheel_link"/>
  <origin xyz="0.0 0.15 0.0"/>
  <axis xyz="0 1 0"/>
</joint>
```



Pipeline симуляции:



4 шага launch-файла симуляции:

1. Запустить Gazebo с миром (world)
2. Загрузить модель робота (spawn_entity)
3. Запустить ros2_control controllers
4. Запустить robot_state_publisher для tf2

Хасро: макрос вместо копипасты

Хасро (XML Macros) — макро-язык для упрощения URDF. Переменные + макросы + include.

Переменная:

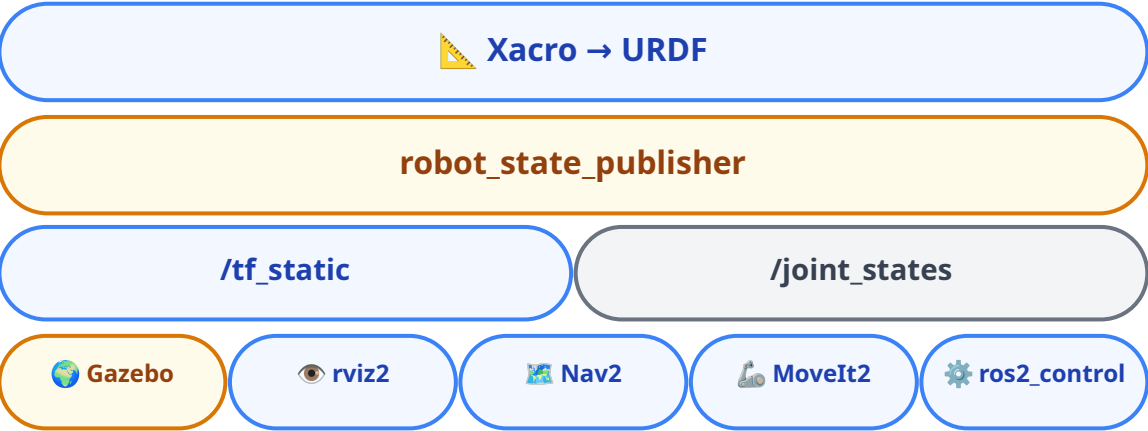
```
<xacro:property name="wheel_radius" value="0.05"/>
```

Макрос для повторяющихся частей:

```
<xacro:macro name="wheel" params="name side">
  <link name="${name}_wheel_link">
    <visual>
      <geometry>
        <cylinder radius="${wheel_radius}" length="0.02"/>
      </geometry>
    </visual>
  </link>
  <joint name="${name}_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="${name}_wheel_link"/>
    <origin xyz="0.0 ${side * 0.15} 0.0"/>
    <axis xyz="0 1 0"/>
  </joint>
</xacro:macro>

<!-- Один вызов — два колеса -->
<xacro:wheel name="left" side="1"/>
<xacro:wheel name="right" side="-1"/>
```

Pipeline: от Хасро до transforms:



5 потребителей URDF:

Потребитель	Что берёт из URDF
rviz2	Геометрию для визуализации
Gazebo	Массы, инерции, коллизии
tf2	Static transforms (через robot_state_publisher)
MoveIt2	Joint-ы, limits, collision
Nav2	Размеры для costmaps

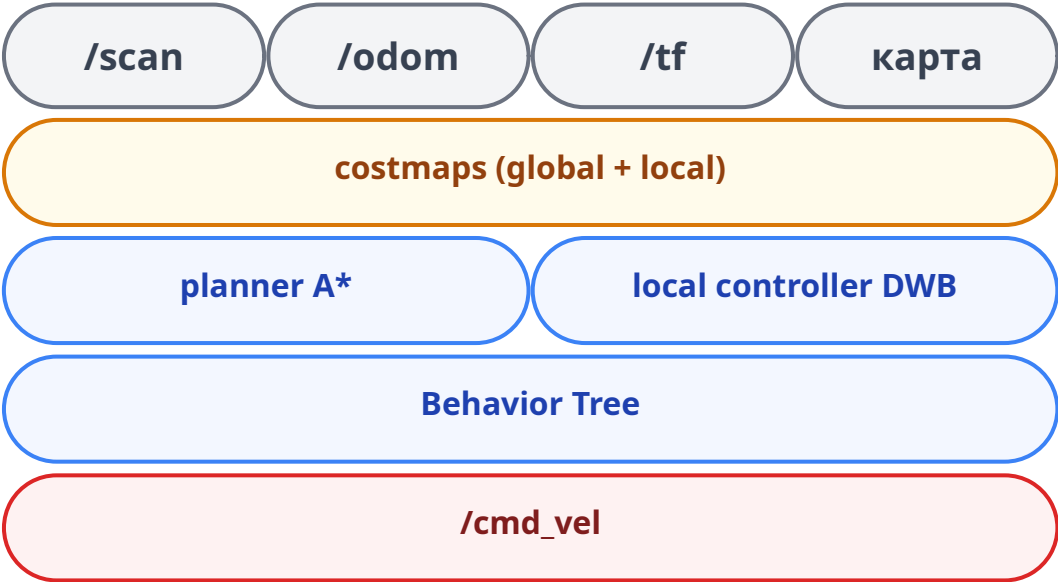
Один макрос — оба колеса. TIAGo: 20+ links, 7-DOF рука полностью, через Хасро с 3 типами базы и 3 типами эндекторов.

Навигационный стек Nav2 — набор узлов, которые вместе решают задачу: доехать из A в B, объезжая препятствия.

Вход: /scan + /odom + /tf + карта
Выход: /cmd_vel + action /navigate_to_pose

Компоненты Nav2:

Компонент	Функция	ROS2-механизм
Global planner	Маршрут A* / Smac	Внутренний
Local controller	Объезд препятствий (DWB)	Внутренний
Costmaps	Карта «стоимости» проезда	Подписка на /scan
Behavior Tree	План → контроль → recovery	Логика bt_navigator
AMCL / SLAM	Локализация на карте	tf2 map→odom



```
from nav2_msgs.action import NavigateToPose
from rclpy.action import ActionClient

client = ActionClient(self,
    NavigateToPose, '/navigate_to_pose')
client.wait_for_server()
goal = NavigateToPose.Goal()
goal.pose.pose.position.x = 2.0
client.send_goal_async(
    goal, feedback_callback=self.feedback)
```


Nav2: SLAM (Simultaneous Localization and Mapping) + Costmap

28

SLAM — строит карту и локализует робота одновременно

```
/scan + /odom → slam_toolbox → /map + tf(map→odom)
```

Costmaps — карты стоимости проезда:

- **Global costmap** — весь маршрут от точки А до В
- **Local costmap** — окрестность робота, реакция на препятствия здесь и сейчас

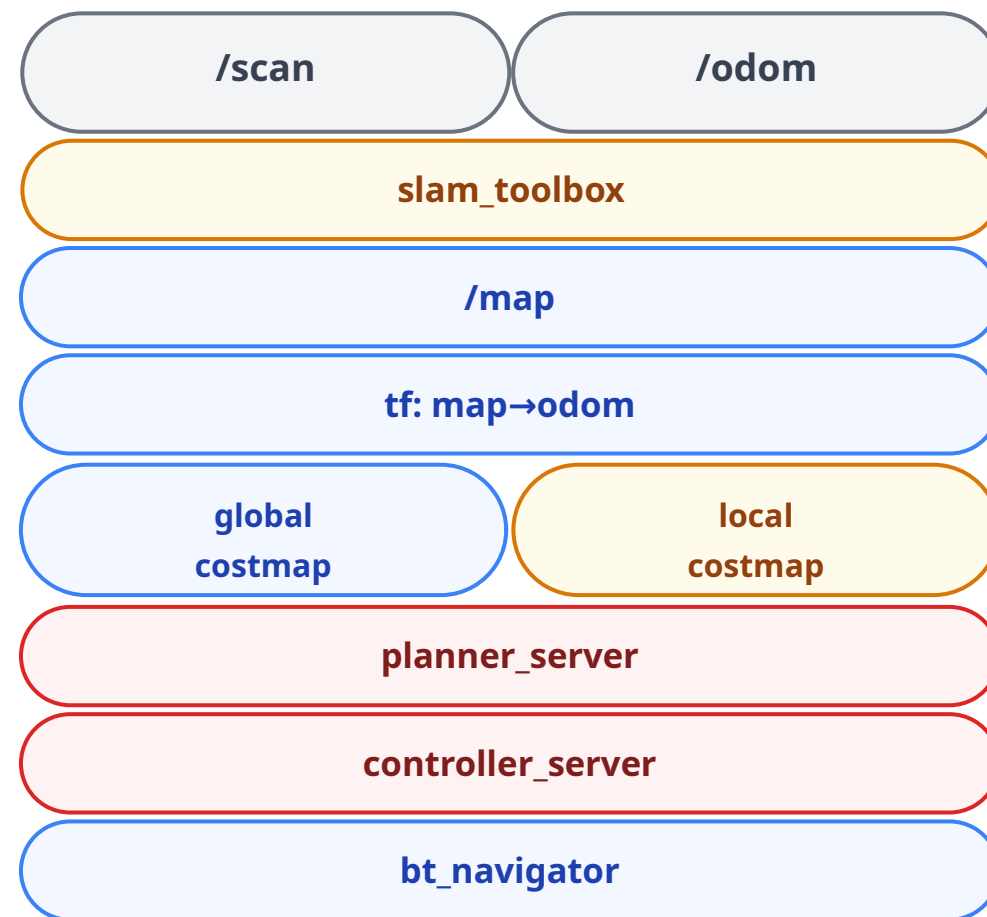
Behavior Tree Nav2 — последовательность:

```
NavigateToPose →  
  ComputePathToPose (global planner) →  
  FollowPath (local controller) →  
  Recovery (если застрял)
```

Все узлы Nav2 — **lifecycle nodes**. Ими управляет **lifecycle_manager**

Уп.2: [knowledge/nav2_bridge.md](#) · [demo/demo5_nav2_architecture.md](#)

Уп.3: [Robot/ 15 карт в pal_maps/](#) · [AMCL](#) · [DWB controller](#)



```
# Запуск Nav2 с картой  
ros2 launch nav2_bringup navigation_launch.py \  
  params_file:=nav2_params.yaml \  
  map:=my_map.yaml
```

MoveIt2: манипуляция

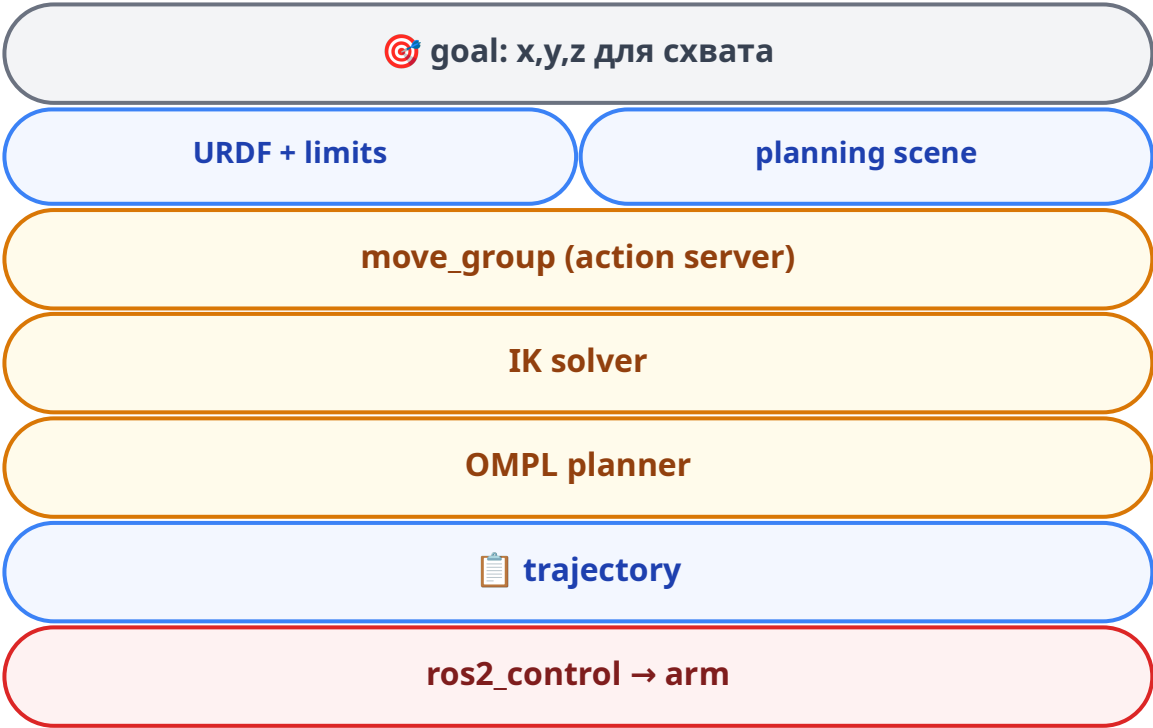
MoveIt2 — планировщик движений манипулятора. Принимает цель (позиция схвата), строит траекторию без столкновений, отправляет в `ros2_control`.

Pipeline: goal → move_group (action server) → IK solver → planner (OMPL) → trajectory → ros2_control → arm

Ключевые компоненты:

Компонент	Что делает
move_group	Центральный action server
IK solver	х,y,z → углы суставов (KDL, TRAC-IK)
Planner	OMPL, STOMP — траектория без столкновений
Planning scene	Мир: столы, стены, объекты
ros2_control	Выполнение траектории на приводах

```
# Запуск
ros2 launch tiago_moveit_config \
  move_group.launch.py
```



```
from moveit_motion.planning_interface import \
    MoveGroupInterface

move_group = MoveGroupInterface(
    self, 'arm', 'robot_description')
goal = PoseStamped()
goal.header.frame_id = 'base_link'
goal.pose.position.x = 0.5
move_group.set_pose_target(goal)
move_group.move() # план + выполнение
```

MoveIt2: IK + Planning Scene

IK (Inverse Kinematics) — пересчитывает позицию схвата в углы суставов:

`x=0.5, y=0.3, z=0.8 → 7 углов: q1..q7`

- Решатели: **KDL** (быстрый), **TRAC-IK** (надёжный)
- Учитывает **joint limits** из URDF

Planning scene — модель окружения:

- Стол, стены, чашка на столе
- Препятствия из `/detections` (YOLO)
- Self-collision — рука не задевает себя

ACM (Allowed Collision Matrix) — в SRDF:

- Какие пары звеньев могут безопасно касаться друг друга
- Ускоряет планирование (не проверять заведомо безопасные пары)

Механизмы ROS2 в MoveIt2:

ROS2	Где в MoveIt2
Action	<code>move_group</code> — action server
Parameters	<code>kinematics.yaml</code> , <code>ompl_planning.yaml</code>
tf2	<code>base_link → arm_1 → ... → gripper</code>
URDF	Модель руки: joints, limits
ros2_control	<code>joint_trajectory_controller</code>

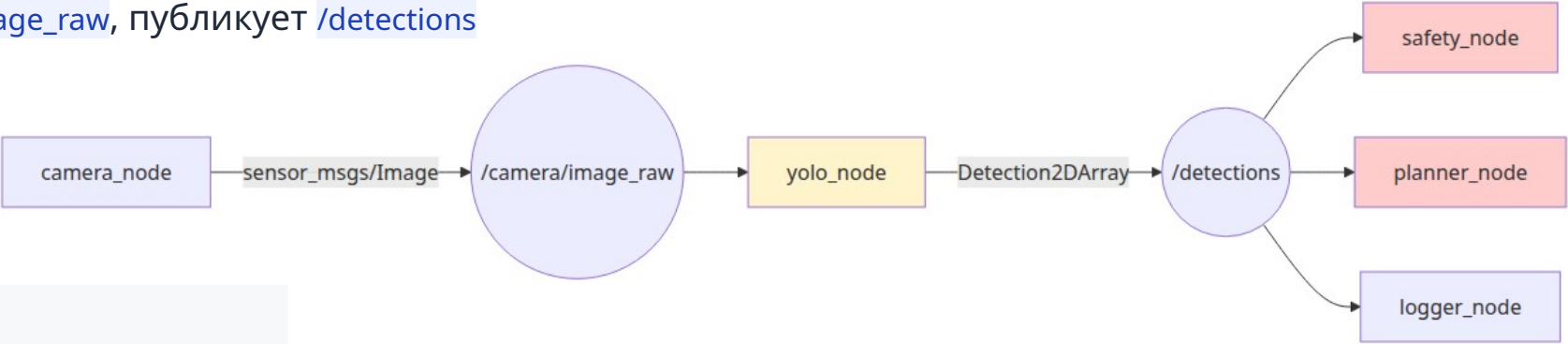
move_group — action server. Тот же паттерн, что и Nav2. Goal → feedback → result.

Ур.2: [knowledge/moveit2_bridge.md](#) — IK, OMPL, планировщики

Ур.3: [Robot/ompl_planning.yaml](#) · [kinematics.yaml](#) · 7-DOF arm

YOLO (You Only Look Once): компьютерное зрение

YOLO-узел — подписан на `/camera/image_raw`, публикует `/detections`



```
class YoloNode(Node):
    def __init__(self):
        super().__init__('yolo_node')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/camera/image_raw',
            self.image_callback, 10)
        self.pub = self.create_publisher(
            Detection2DArray, '/detections', 10)
        self.model = load_yolo_model('yolov8n.pt')

    def image_callback(self, msg):
        cv_img = self.bridge.imgmsg_to_cv2(msg)
        results = self.model(cv_img)
        det_msg = Detection2DArray()
        det_msg.header = msg.header
        det_msg.detections = self._convert(results)
        self.pub.publish(det_msg)
```

Компоненты YOLO в ROS2:

Компонент	Назначение
cv_bridge	ROS Image ↔ numpy.ndarray
sensor_msgs/Image	Изображение в ROS2
vision_msgs/Detection2DArray	Результаты детекции

Граница безопасности:

Что YOLO делает	Что YOLO НЕ делает
Публикует bounding boxes	Не управляет <code>/cmd_vel</code>
Публикует confidence	Не вызывает <code>/navigate_to_pose</code>
Работает как perception	Не принимает решений о движении

YOLO публикует факты. НЕ управляет движением. Если YOLO ошибся — последствия ограничены лишним bounding box.

LLM Bridge (Large Language Model): голосовое управление

LLM bridge — переводит голосовую команду в high-level action работа

Pipeline: голос → STT → LLM → parser → policy → safety → action

```
# Вариант 1: внешний API (OpenAI)
response = requests.post(
    'https://api.openai.com/v1/chat/completions',
    json={'model': 'gpt-4o-mini',
          'messages': [
              {'role': 'system', 'content': SYSTEM_PROMPT},
              {'role': 'user', 'content': msg.data}]}])
cmd = json.loads(response.json()
    ['choices'][0]['message']['content'])
# → {"action": "navigate", "params": {"x": 3.0, "y": 1.5}}

# Вариант 2: локальная модель (Ollama)
# → http://localhost:11434/api/chat

# Вариант 3: embedded (llama-cpp-python)
# → Llama(model_path="...gguf")
```



Интеграция	Плюсы	Минусы
API (OpenAI)	Не нужна GPU	Нужен интернет, задержка 1-3с
Ollama (локально)	Без интернета	Нужен 8+ GB RAM
Embedded (llama.cpp)	Минимальная задержка	До 4 GB RAM

Маршрут работы с материалом

Уровень 1 — Понимание (лекция 80 мин)

- ROS Graph, Node, Executor
- Topic, Service, Action — когда что
- Launch, params, tf2, QoS
- Simulation-first, Nav2, MoveIt2
- **Результат:** читает ROS Graph, пишет pub/sub

Уровень 2 — Самостоятельная работа

```
2_knowledge/  
2_practice/  
01_workspace → 02_package → 03_nodes  
→ 04_pub_sub → 05_services → 06_actions  
→ 07_launch → 08_mini_project
```

- Каждая тема: readme + код + check
- **Результат:** собирает workspace, пишет узлы, запускает launch

Уровень 3 — Архитектурный кейс TIAGo

Параметр	Значение
Пакетов	64 (18 репозиторийев)
Launch-файлов	56
YAML-конфигов	170+
TF2 frames	25+
Subsystems	8
DOF руки	7
Сенсоров	LiDAR + RGB-D + IMU

- **Результат:** читает чужой ROS2-проект, понимает связи

Среда: Ур.2: [.devcontainer/ \(Jazzy\)](#) · Ур.3: [Robot/.devcontainer/ \(Humble\)](#)

ROS2
сегодня

Сенсорика

Цифровые
двойники

YOLO

SLAM
Nav2

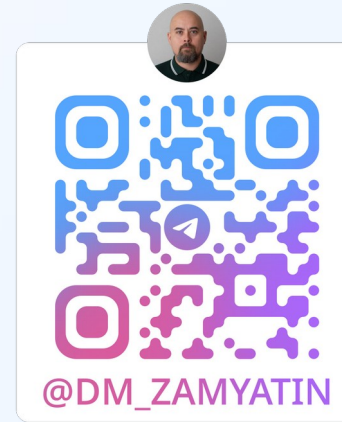
Манипуляторы

VLA

Sim-to-real

M2M

Безопас-
ность



Спасибо за внимание!

После лекции у вас уже точно есть представления о модульной архитектуре ROS2
и есть инструменты для создания собственного распределенного робототехнического приложения

GitHub: https://github.com/DiZamer/ROS2_lecture_ya_camp.git

